Empirical Study on the Productivity of the Pair Programming

Gerardo Canfora, Aniello Cimitile, and Corrado Aaron Visaggio

Research Centre on Software Technology,
University of Sannio, viale Traiano 1, Palazzo ex-Poste, 82100 Benevento, Italy
{canfora,cimitile,visaggio}@unisannio.it

Abstract. Many authors complain the lack of empirical studies which assess the benefits and the drawbacks of agile practices. Particularly, pair programming rises an interesting question for managers: does pair programming mean to pay two developers in the place of one? We realized an experiment to compare the productivity of a developer when performing pair programming and when working as solo programmer. We obtained empirical evidence that pair programming decreases the effort of the single programmer.

Introduction

Pair programming is an agile practice [9] consisting of two developers working at the same code, side by side on the same machine: one develops the code, the other one reviews it. A number of benefits are enumerated in literature, such as:

- ***Pair pressure.*** Working in pairs drives the developer out of his personal 'comfort zone', and requires a major involvement, humility, and awareness [20].
- ***Economics.*** The continuous review of code while writing it, increases sensitively the rate of defects' removal [19].
- ***Satisfaction.*** Working in pairs increases enjoyment, and then commitment of the developers [17].
- ***Knowledge transfer.*** Pair working should increase the knowledge transfer among pair's members [11].
- ***Team building and communication.*** Developers learn to discuss, to communicate, and generally to work in group [20].

At our best knowledge, there is not a mature body of knowledge that validates these conjectures with empirical analyses, although some valuable studies have been accomplished in this direction, as the section Related Works shows. As a matter of fact, some authors remark [1, 10, 21] that there is a need for more case studies and field experiments, possibly with industries, in order to define the actual effects of this practice on the software development process.

Economics, in particular, is the focus of this paper: it is immediate to wonder whether pair programming means to sustain the cost of two developers for the same work that a developer could perform. Some previous papers discuss this concern, and outline that pair programming decreases the effort with respect to a solo programmer with the additional benefit to increase also the quality of the code produced.

The novelty of our study stands in the method of investigation that we have adopted. We analyze the effects of pair programming on each single programmer, by

H. Baumeister et al. (Eds.): XP 2005, LNCS 3556, pp. 92–99, 2005.

comparing the performances when the developer programs in pair and when the same developer programs as solo. Conversely, literature reports researches in which the outcomes of pairs are compared with those of solo programmers, but, at our best knowledge, there is not consolidated results on the effect of pair programming on the performances of an individual programmer.

We realized an experiment in order to answer the research goal, that can be formulated, in accordance to the GQM paradigm [2] as follows: *Analyze* pair programming sessions *for the purpose of* evaluating *with respect to* its capability of reducing the developing time of each single programmer *from the point of view of* the researchers *in the context of* students' groups with different degrees.

The experiment is part of a larger research program that investigates not only the performances allowed by the practice of pair programming, but also: the quality of the code produced; the relationship between pair programming and knowledge [6]; and finally, how pair programming can improve the systems design[5, 7].

The paper continues as follows: the section *Related Work* reviews relevant literature and outlines the distinctive aspects of our work; the section *Experimental Design* describes the experiment; the section *Data Analysis* and the section *Statistical Tests* provide a commentary for the data obtained as results of the experiment; and, finally, the *Conclusion*'s section draws the observation and discusses the future work.

Related Work

The current work is part of a family of experiments, aiming at evaluating how pair programming affects effort and productivity of each programmer with respect to the solo programming.

When the term 'pair programming' was not yet widespread, Nosek investigated Collaborative Programming [16]. Collaborative Programming means 'two programmers working jointly on the same algorithm and code'. Basically, it was a form of pair programming *ante-litteram*. Nosek executed an experiment with experienced programmers and it showed that collaborative programmers outperformed the individual programmers. Interestingly, a secondary observation stemmed from the experiment: collaborative programmers reported higher values of enjoyment of the process and confidence about their work. With the growing interest for pair programming in the software engineering research community, more focalized investigations have been published. However, initially the attention of researchers focussed mainly on quality and productivity. In [19] the authors realized a structured experiment in order to understand the difference between pair and solo programming. The results showed that the time was reduced up to the 50%. The authors found other outcomes, concerning quality and enjoyment of the participants. In [15], the authors compared the pair programming with the Personal Software Process proposed by Humprey. They found that pair programming can decrease the development time but not up the 50% reported by Williams in[19]; pair programming is more predictable than solo programming in terms of development time; and the amount of rework was reduced with the pair programming. In [3] the authors investigate the performance of pair programming when the components of the pair are not co-located: they found that distributed pair programming seems to be comparable to colocated software development in terms of the productivity and quality. Productivity is measured as lines of code per hour.